

**PRN:** 2124UCSM1057

**Class:** 3rd Year

**Department:** Cyber Security

# AWS IAM Least-Privilege Assignment

## Aim

To design and implement a rigorous AWS Identity and Access Management (IAM) structure that enforces the Principle of Least Privilege, consisting of:

- Distinct IAM Groups — **Admins, Developers, Auditors** — each with an attached custom policy.
- Multi-Factor Authentication (MFA) enforced for the Admins group.
- A custom IAM Role allowing an EC2 instance to access S3 securely, without hardcoded credentials.

## Tools Used

- **Terraform** (Infrastructure as Code) to define and provision the IAM structure in a reviewable, version-controlled way.
- **AWS Identity and Access Management (IAM).**

## Project Structure

|                |                                                 |
|----------------|-------------------------------------------------|
| terraform/     |                                                 |
| main.tf        | Provider and Terraform configuration            |
| variables.tf   | Region, project name, target S3 bucket ARN      |
| groups.tf      | The three IAM groups                            |
| policies.tf    | Custom least-privilege policies per group       |
| ec2_s3_role.tf | EC2 instance role, policy, and instance profile |
| outputs.tf     | Useful ARNs and names after apply               |

## 1. IAM Groups

Three groups are created to separate duties cleanly:

| Group      | Purpose                                      |
|------------|----------------------------------------------|
| Admins     | Full account administration                  |
| Developers | Day-to-day application development on EC2/S3 |
| Auditors   | Read-only compliance and security review     |

```
resource "aws_iam_group" "admins" {
  name = "${var.project_name}-Admins"
  path = "/"
}
```

```

resource "aws_iam_group" "developers" {
  name = "${var.project_name}-Developers"
  path = "/"
}

resource "aws_iam_group" "auditors" {
  name = "${var.project_name}-Auditors"
  path = "/"
}

```

## 2. Custom Policies (Principle of Least Privilege)

### 2.1 Admins — full access, gated by MFA

IAM groups cannot natively require MFA, so this is implemented with two attached policies:

- **AdminPermissions** — grants administrative access, but every statement carries a condition on `aws:MultiFactorAuthPresent`, so the permissions only take effect in an MFA-authenticated session.
- **EnforceMFA** — denies every action *except* a small allow-list (viewing one's own IAM user, registering/enabling an MFA device, changing one's own password) whenever MFA has not been presented.

```

statement {
  sid      = "AdminFullAccessWhenMFA"
  effect   = "Allow"
  actions  = ["*"]
  resources = ["*"]

  condition {
    test      = "BoolIfExists"
    variable  = "aws:MultiFactorAuthPresent"
    values    = ["true"]
  }
}

```

**Result:** an Admin without MFA configured can log in and register a device — and nothing else. Once MFA is active, full admin access applies.

### 2.2 Developers — scoped to the job, not the account

- Full EC2 lifecycle actions (RunInstances, Start/Stop/Reboot/Terminate, describe calls), restricted to the deployment region.
- GetObject / PutObject / ListBucket on **one named application S3 bucket only** — not `arn:aws:s3:::*`.
- CloudWatch Logs access scoped to a single log-group prefix.
- An explicit **Deny** on `iam:*` and billing/organization actions, as a defense-in-depth backstop.

### 2.3 Auditors — strictly read-only

Only Get\* / List\* / Describe\* actions across IAM, EC2, S3, CloudTrail, AWS Config, CloudWatch, and Cost Explorer, backed by an explicit **Deny** on any mutating verb. No statement in this policy can create, modify, or delete a resource.

### 3. Custom IAM Role — EC2 Access to S3 (No Hardcoded Credentials)

A dedicated role lets an EC2 instance read and write objects in a specific S3 bucket without ever storing an access key on the instance.

**Trust policy** — only the EC2 service may assume the role:

```
statement {
  effect = "Allow"
  actions = ["sts:AssumeRole"]

  principals {
    type       = "Service"
    identifiers = ["ec2.amazonaws.com"]
  }
}
```

**Permissions policy** — scoped to the bucket, not account-wide S3 access:

```
statement {
  sid      = "ListSpecificBucket"
  effect   = "Allow"
  actions  = ["s3:ListBucket"]
  resources = [var.s3_bucket_arn]
}

statement {
  sid      = "ReadWriteObjectsInBucket"
  effect   = "Allow"
  actions  = ["s3:GetObject", "s3:PutObject", "s3:PutObjectAcl"]
  resources = ["${var.s3_bucket_arn}/*"]
}
```

The role is wrapped in an **instance profile**, which is what actually attaches to the EC2 instance. Once attached, the instance retrieves short-lived, automatically rotated credentials from the Instance Metadata Service (IMDS) — no static access key/secret is stored on disk or in application configuration.

### 4. Deployment

```
cd terraform
terraform init
terraform plan -var="s3_bucket_arn=arn:aws:s3:::your-actual-bucket"
terraform apply -var="s3_bucket_arn=arn:aws:s3:::your-actual-bucket"
```

To attach the role to an EC2 instance:

```
resource "aws_instance" "app" {
  # ...
  iam_instance_profile = aws_iam_instance_profile.ec2_s3_access.name
}
```

### 5. Conclusion

This structure demonstrates the Principle of Least Privilege in practice: each group and role is granted only the permissions required for its function, resource access is scoped to specific ARNs rather

than left open with \*, and Admin access is additionally gated behind MFA. The EC2 role removes the need for any hardcoded AWS credentials by relying on an IAM role and instance profile instead.